

Даже с простым образом мы уже имели дело с множеством опций командной строки как при сборке, пуше, так и при запуске образа.

Далее мы перейдем к инструменту под названием [Docker Compose](#) для управления ими. Docker Compose раньше был отдельным инструментом, но теперь он интегрирован в Docker и может использоваться как и остальные команды Docker.

Docker Compose разработан для упрощения запуска многоконтейнерных приложений с помощью одной команды.

Предположим, что мы находимся в папке, где у нас есть наш Dockerfile со следующим содержимым:

```
FROM ubuntu:22.04

WORKDIR /mydir

RUN apt-get update && apt-get install -y curl python3
RUN curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o
/usr/local/bin/yt-dlp
RUN chmod a+x /usr/local/bin/yt-dlp

ENTRYPOINT ["/usr/local/bin/yt-dlp"]
```

Давайте теперь создадим файл с именем `docker-compose.yml` :

```
version: '3.8'

services:
  yt-dlp-ubuntu:
    image: <username>/<repositoryname>
    build: .
```

Настройка версии не очень строгая, она просто должна быть выше 2, потому что иначе синтаксис значительно отличается. См. <https://docs.docker.com/compose/compose-file/> для получения дополнительной информации.

Значение ключа `build` может быть путем в файловой системе (в примере это текущая директория `.`) или объектом с ключами `context` и `dockerfile`, см. [документацию](#) для получения дополнительной информации.

Теперь мы можем собирать и пушить всего этими командами:

```
$ docker compose build
$ docker compose push
```

Томы в Docker Compose

Чтобы запустить образ, как мы делали ранее, нам нужно добавить bind mounts томов. Томы в Docker Compose определяются следующим синтаксисом `location-in-host:location-in-container`. Compose может работать без абсолютного пути:

```
version: '3.8'

services:

  yt-dlp-ubuntu:
    image: <username>/<repositoryname>
    build: .
    volumes:
      - ./mydir
    container_name: yt-dlp
```

Мы также можем дать контейнеру имя, которое он будет использовать при запуске с помощью `container_name`. Имя сервиса можно использовать для его запуска:

```
$ docker compose run yt-dlp-ubuntu https://imgur.com/JY5tHqr
```

Упражнение 2.1

:::info Упражнение 2.1

Давайте теперь воспользуемся Docker Compose с простым веб-сервисом, который мы использовали в [Упражнении 1.3](#)

Без команды `devopsdockeruh/simple-web-service` будет создавать логи в свой `/usr/src/app/text.log`.

Создайте файл `docker-compose.yml`, который запускает `devopsdockeruh/simple-web-service` и сохраняет логи в вашу файловую систему.

Отправьте `docker-compose.yml` и убедитесь, что он работает просто запуском `docker compose up`, если файл логов существует.

:::

Веб-сервисы в Docker Compose

Compose действительно предназначен для запуска веб-сервисов, так что давайте перейдем от простых бинарных оберток к запуску HTTP сервиса.

<https://github.com/jwilder/whoami> — это простой сервис, который выводит текущий id контейнера (hostname).

```
$ docker container run -d -p 8000:8000 jwilder/whoami
736ab83847bb12ddd8b09969433f3a02d64d5b0be48f7a5c59a594e3a6a3541
```

Перейдите с помощью браузера или `curl` на `localhost:8000`, они оба ответят `id`.

Остановите контейнер, чтобы он не блокировал порт 8000.

```
$ docker container stop 736ab83847bb
```

```
$ docker container rm 736ab83847bb
```

Давайте создадим новую папку и файл Docker Compose `whoami/docker-compose.yml` из опций командной строки.

```
version: '3.8'

services:
  whoami:
    image: jwilder/whoami
    ports:
      - 8000:8000
```

Протестируйте его:

```
$ docker compose up -d
$ curl localhost:8000
```

Переменные окружения также могут быть переданы контейнерам в Docker Compose следующим образом:

```
version: '3.8'

services:
  backend:
    image:
    environment:
      - VARIABLE=VALUE
      - VARIABLE2=VALUE2
```

Обратите внимание, что есть также [другие](#), возможно, более элегантные способы определения переменных окружения в Docker compose.

Упражнения 2.2 - 2.3

:::info Упражнение 2.2

Прочитайте, как добавить команду в `docker-compose.yml` из [документации](#).

Знакомый образ `devopsdockeruh/simple-web-service` можно использовать для запуска веб-сервиса, см. [упражнение 1.10](#).

Создайте `docker-compose.yml` и используйте его для запуска сервиса так, чтобы вы могли использовать его с вашим браузером.

Отправьте `docker-compose.yml` и убедитесь, что он работает просто запуском `docker compose up`

:::

:::caution Обязательное Упражнение 2.3

Как мы видели ранее, запуск приложения с двумя программами был нетривиальным, и команды получились немного длинными.

В [предыдущей части](#) мы создали Dockerfiles для [frontend](#) и [backend](#) примерного приложения. Далее, упростите использование в один docker-compose.yml.

Настройте backend и frontend из [части 1](#) для работы в Docker Compose.

Отправьте docker-compose.yml

...